Dictionary Attack Lab

This lab builds a three-node VirtualBox topology (Kali attacker, Ubuntu target with OpenSSH and a simple PHP/SQLite web app, and Windows Server target with RDP) to demonstrate password attack techniques and mitigations. Using Hydra, the attacker performs online dictionary attacks against SSH, RDP and the web login form; using Impacket/secretsdump and Hashcat, NTLM hashes are extracted and cracked offline. The defensive phase implements Fail2Ban, local account lockout policies, Network Level Authentication (NLA) and firewall restrictions; the report compares attack success rates before and after hardening and discusses why offline hash-cracking remains a risk.

PC specifications:

RAM	16GB
Storage	512 SSD
Processor	I5 10th gen (10300H)
Cores	4

Virtualization:

Virtualization is done on Oracle VirtualBox. The description of the VMs are given below.

Kali Linux (named kali linux in the VirtualBox)	BaseMemory: 6140MB Processor: 2
Windows Server (Named Windows Server 2022 in VirtualBox)	BaseMemory: 5632 MB Processor: 2
Ubuntu Server 24.03 (Named Ubuntu Server in VirtualBox)	BaseMemory: 2560 MB Processor: 1

Network Configuration in each of the VM's:

Windows Server 2022	IP Address: 192.168.1.10 Subnet Mask:255.255.255.0
Kali Linux	IP Address: 192.168.1.20 Subnet Mask:255.255.255.0
Ubuntu Server	IP Address: 192.168.1.30 Subnet Mask:255.255.255.0

Be sure to use a host only adapter on all the VM as it enables them to communicate with each other. After the configuration, ping each other to make sure if communication actually works.

Bruteforcing Ubuntu Server using SSH and the WebApp which is hosted in there using hydra tool.

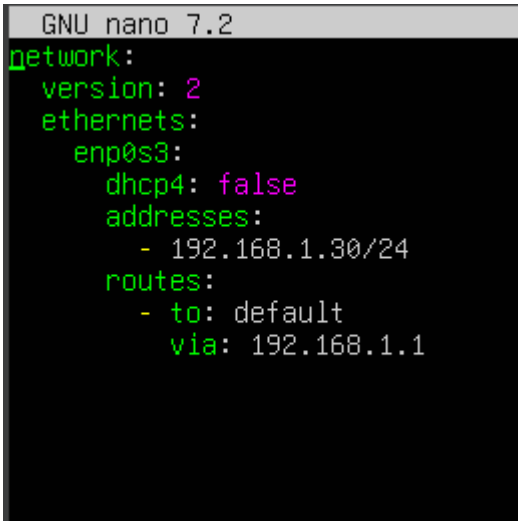
Hydra — a fast, parallelized network login cracker used in penetration testing to perform online dictionary and brute-force attacks against services (SSH, RDP, FTP, HTTP forms, etc.) to evaluate authentication strength.

To bruteforce the ubuntu server , first we have to have OpenSSH.
OpenSSH will be asked when you are installing Ubuntu for the first time.

After that you have to set up a static IP address. To do that, enter this command

Sudo nano /etc/netplan/(the_filename_in_in_directory)

You will be prompted with a window and modify it as show below.



After modifying the file, execute this command.

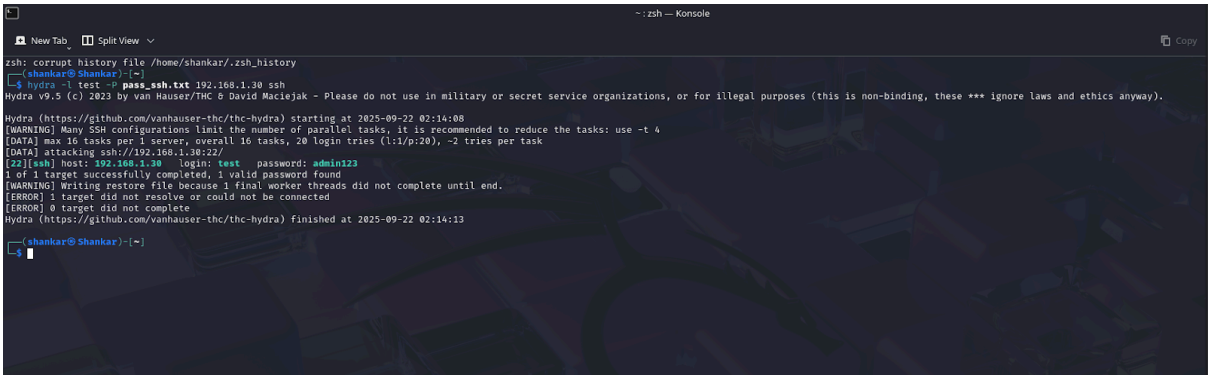
sudo netplan apply

This command basically applies the changes you made on the file.

Now its time to attack. Go to kali linux and pull up the terminal and type this command.(Structure of the attacking command)

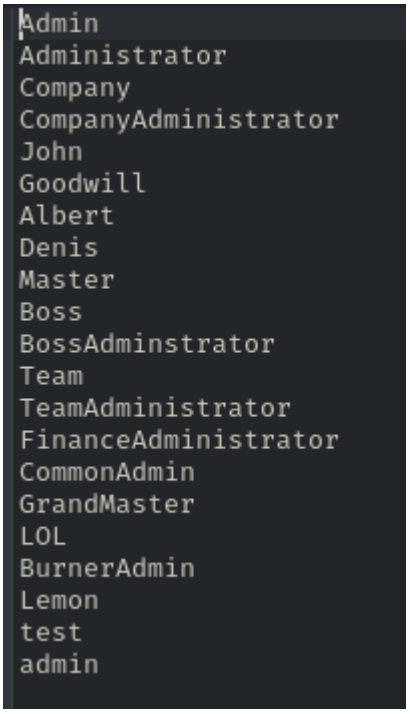
Hydra -L custom_user_ wordlist -P custom_pass_wordlist ser_ip ssh.

Now it should look something like this:

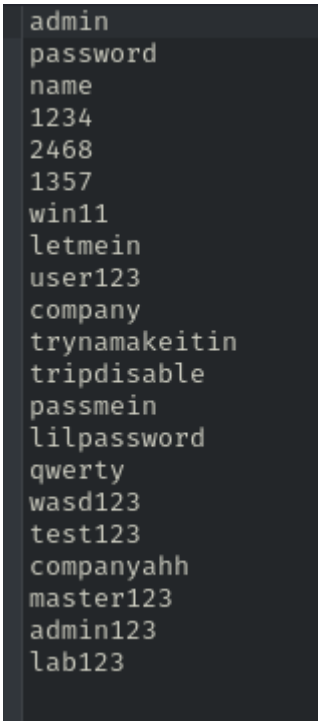


The wordlist which i am going to provide will be the same for the webapp too.

User Wordlist:



Password Wordlist:



Now the next attack, which we need to host a Webapp. To do that we have to install apache server and sqlite3. Here are the commands.

sudo apt install -y php libapache2-mod-php php-sqlite3

sudo apt install -y apache2 sqlite3

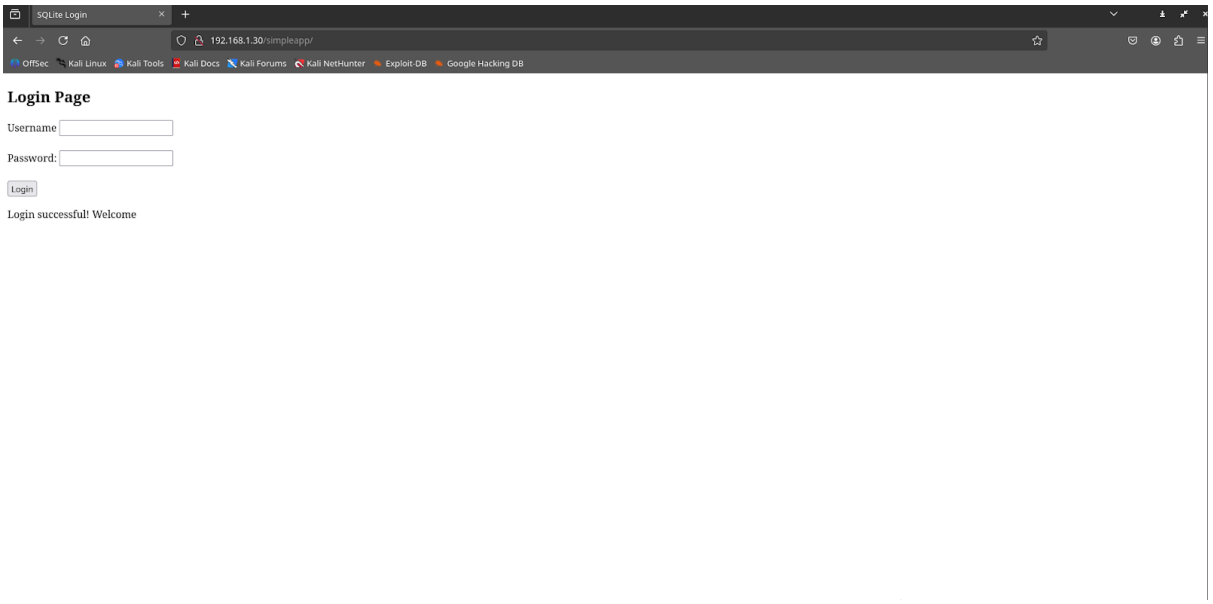
And then create a directory in the apache server named simpleapp and create a filename. The commands for the following words.

Sudo nano /var/www/html/simpleapp/index.php

Now write a code like this:

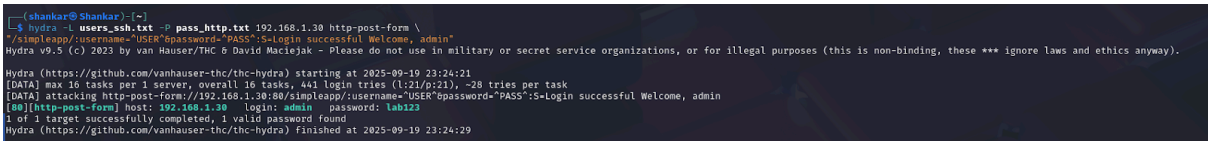
```php
<?php
session_start();
$message='';
ini_set('display_errors',1);
ini_set('display_startup_errors',1);
error_reporting(E_ALL);

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $db = new SQLite3(__DIR__.'/users.db');
    $stmt = $db->prepare('SELECT * FROM users WHERE username = :username AND password = :password');
    $stmt->bindValue(':username', $_POST['username']);
    $stmt->bindValue(':password', $_POST['password']);
    $result = $stmt->execute();
    if ($result->fetchArray()) {
        $message = "Login successful Welcome, " . htmlspecialchars($_POST['username']);
    } else {
        $message = "Invalid username or password";
    }
}
?>

<!DOCTYPE html>
<html>
<head><title>Lab Login</title></head>
<body>
<h2>Login Page</h2>
<form method="POST">
    <label>Username:</label>
    <input type="text" name="username" required><br><br>
    <label>Password:</label>
    <input type="password" name="password" required><br><br>
     <input type="submit" value="Login">
</form>
<p><?php echo $message; ?></p>
</body>
</html>
```

The webapp will look like this:



Create a users.db in the same directory and add what you want in that, we especially need user credentials.
The credentials I gave as
User: admin
Password: lab123

Now it's time to attack the webapp page,



I was able to successfully get the credentials by using the hydra tool.

Defending ubuntu server against the bruteforce using fail2ban tool

Fail2Ban — a lightweight intrusion-prevention tool that monitors log files for repeated failed login attempts and automatically blocks offending IPs (via firewall rules) to stop brute-force attacks.

To install fail2ban tool,

sudo apt install fail2ban -y

After installing fail2ban, go the fail2ban directory

Cd /etc/fail2ban

Now copy the jail.config file inside it as this file must not be altered or tampered with.

sudo cp jail.config jail.local

Now go into the jail.local file and you will see that the ssh will defaultly be listed there.

After enabling the apache server and fail2ban by using the command,

Sudo systemctl enable apache2
Sudo systemctl start apache2
Sudo systemctl start fail2ban

Now launch the attack from the kali linux and the result will be displayed on the below screenshot:

```
┌──(shankar㉿Shankar)-[~]
└─$ hydra -l test -P pass_ssh.txt 192.168.1.30 ssh
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-20 04:51:43
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 20 login tries (l:1/p:20), ~2 tries per task
[DATA] attacking ssh://192.168.1.30:22/
[ERROR] could not connect to ssh://192.168.1.30:22 - Connection refused
```

Now if you see the status of the fail2ban client of the ssh you will see something like this and so with the log files.

```
test@shankar:/etc/fail2ban$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 1
|  |- Total failed:     36
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 1
   |- Total banned:     2
   `- Banned IP list:   192.168.1.20
test@shankar:/etc/fail2ban$
```

```
test@shankar:/etc/fail2ban$ sudo tail -f /var/log/fail2ban.log
2025-09-20 11:16:37,422 fail2ban.filter         [3246]: INFO    [sshd] Found 192.168.1.20 - 2025-09-20 11:16:37
2025-09-20 11:16:37,422 fail2ban.filter         [3246]: INFO    [sshd] Found 192.168.1.20 - 2025-09-20 11:16:37
2025-09-20 11:16:37,422 fail2ban.filter         [3246]: INFO    [sshd] Found 192.168.1.20 - 2025-09-20 11:16:37
2025-09-20 11:16:37,422 fail2ban.filter         [3246]: INFO    [sshd] Found 192.168.1.20 - 2025-09-20 11:16:37
2025-09-20 11:16:37,422 fail2ban.filter         [3246]: INFO    [sshd] Found 192.168.1.20 - 2025-09-20 11:16:37
2025-09-20 11:16:37,423 fail2ban.filter         [3246]: INFO    [sshd] Found 192.168.1.20 - 2025-09-20 11:16:37
2025-09-20 11:16:37,423 fail2ban.filter         [3246]: INFO    [sshd] Found 192.168.1.20 - 2025-09-20 11:16:37
2025-09-20 11:16:37,621 fail2ban.actions        [3246]: NOTICE  [sshd] 192.168.1.20 already banned
2025-09-20 11:16:37,621 fail2ban.actions        [3246]: NOTICE  [sshd] 192.168.1.20 already banned
2025-09-20 11:16:37,621 fail2ban.actions        [3246]: NOTICE  [sshd] 192.168.1.20 already banned
```

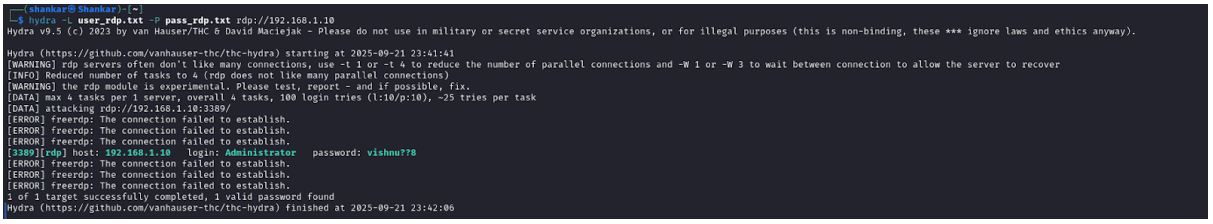
Bruteforcing windows server rdp credentials by using hydra:

To do this attack, make sure RDP is enabled . And be sure to remove the rdp connection restriction from the firewall as it disrupts making a connection to it.
Use this command in the command line to lift the restricion,

netsh advfirewall firewall set rule group="remote desktop" new enable=yes

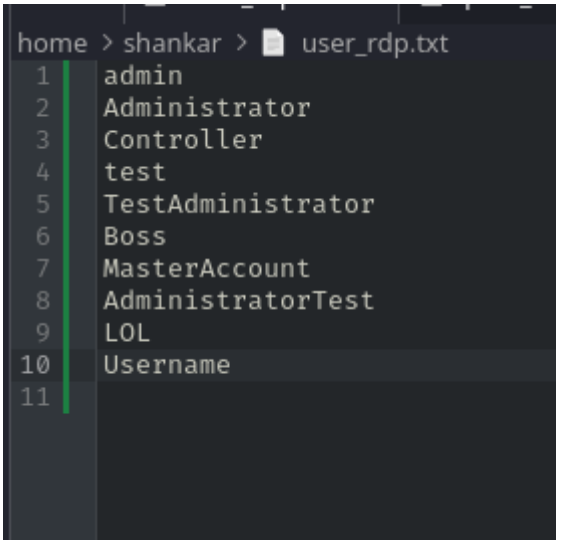
The credentials for the account used here are
User: Administrator
Password : vishnu??8

Now launch an attack from the kali linux machine and the output will look like this,

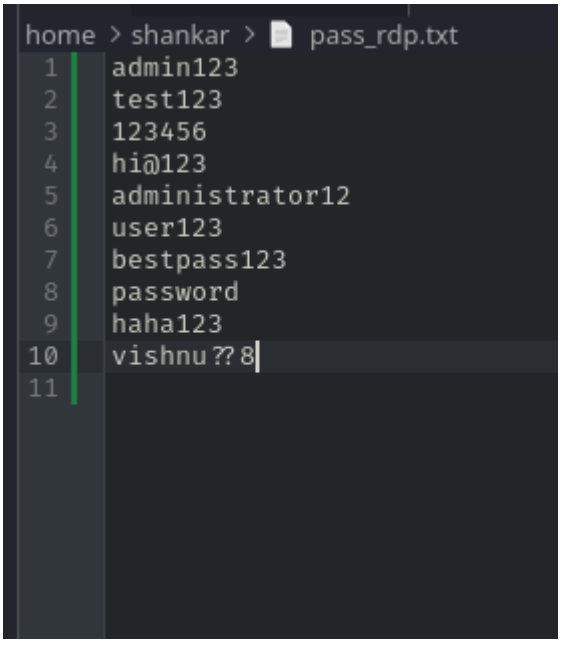


The wordlists used here are

User Wordlist:



Password Wordlist:



Defensive methods against this attack

To save yourself from this attack, you gotta take these precautions by

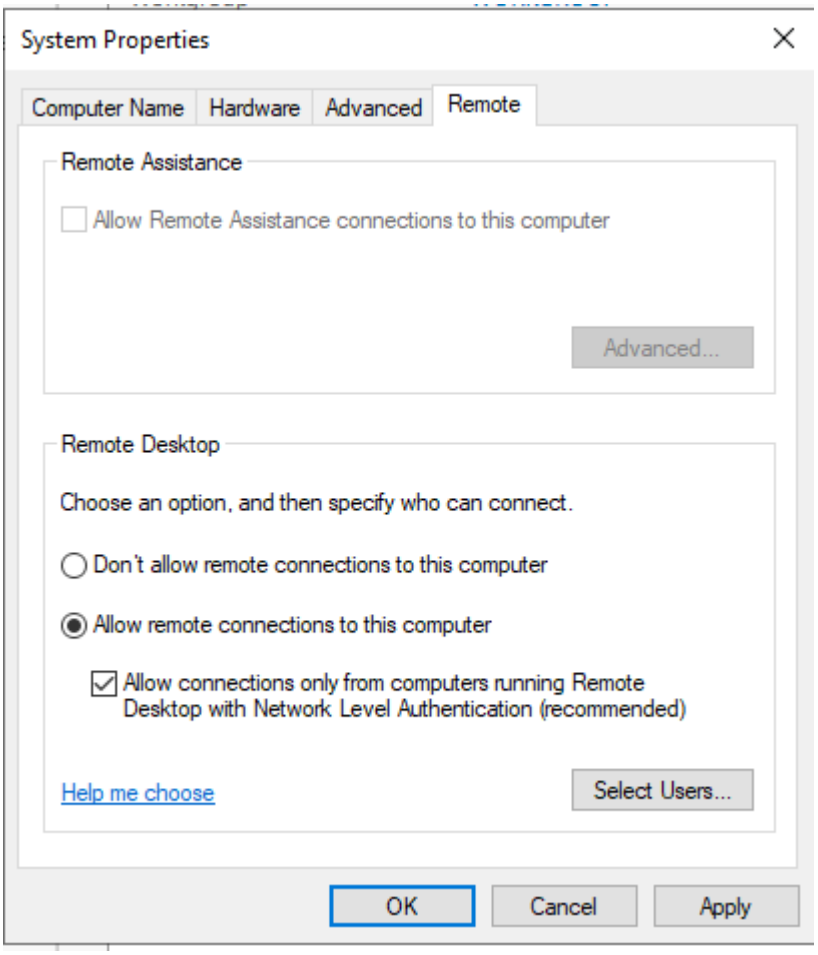
Implementing account lockout:

net accounts /minpwlen:12 /uniquepw:5 /maxpwage:90

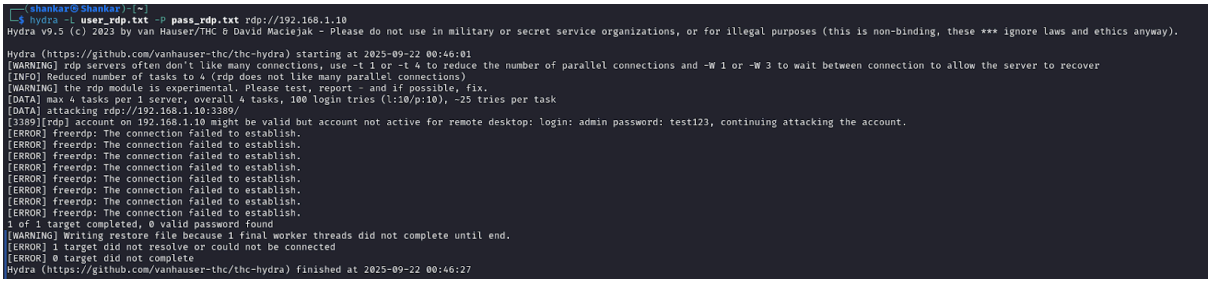
Apply account lockout:

net accounts /lockoutthreshold:3 /lockoutduration:30 /lockoutwindow:30

Apply NLA:



And the final result will be like this:



Exporting hashes from a VM and attempting to crack it using HashCat:

About HashCat:

A utility used to generate, verify, and analyze cryptographic hashes for files or passwords.

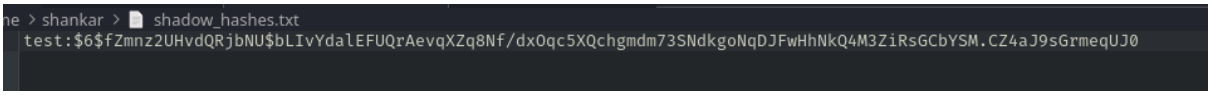
In order to crack the hash file, first we have to generate a hash file . I am gonna generate it on the ubuntu server. To do that,

sudo mkdir -p /root/lab_hashes && sudo chmod 700 /root/lab_hashes

sudo awk -F: '($2!="" && $2!~"^*" && $2!~"^!"){print $1 ":" $2}' /etc/shadow | sudo tee /root/lab_hashes/shadow_hashes.txt >/dev/null

sudo chmod 600 /root/lab_hashes/shadow_hashes.txt

The final hash generated:



Now its time to export it to kali . To do that, we gotta use these commands.

scp test@192.168.1.30:/root/lab_hashes/shadow_hashes.txt ~/lab_hashes_shadow.txt /home/shankar

The file will be found in the directory you mentioned in this.

Now time for hashcat attack,

I couldnt crack this file for some reason, but this is a way to crack a hash file.

HashCat vs Hydra:

Purpose

- **Hashcat** — an *offline* password-recovery tool. It takes password hashes (e.g. `/etc/shadow` style) and attempts to recover the plaintext by trying candidate passwords. It is optimized for high-throughput cracking using GPUs and supports many hash formats and attack modes (dictionary, mask, combinator, rules, hybrid).

- **Hydra** — an *online* network authentication brute-forcer. It attempts logins against network services (SSH, FTP, HTTP forms, SMB, RDP proxies, etc.) by making authentication requests to the target host(s). It uses CPU/network concurrency to try many username/password combinations against live services.

Strengths & tradeoffs

- **Hashcat**
    - Strengths: Extremely fast for supported hash types (GPU-accelerated), many attack modes and tuning options, good for large-scale offline audits where you possess hashes legitimately.
    - Tradeoffs: Requires the hash; cracking times vary widely depending on hash algorithm and password

strength. GPU setup and drivers required for top performance.

- **Hydra**
    - Strengths: Useful for testing live authentication endpoints and password spraying / credential-stuffing scenarios. Supports many protocols and authentication methods.
    - Tradeoffs: Constrained by network latency and remote service rate limits; high request rates can lock accounts, trigger IDS/IPS, or get you blocked. Slower per-try than GPU-powered offline cracking.

Typical use-cases

- **Use Hashcat when:** You have hashed credentials obtained with authorization (lab / incident response) and want to recover plaintext passwords or test the effectiveness of password policies.
- **Use Hydra when:** You need to verify how a live service behaves under password-guessing attempts (lockout policy, rate limiting, account lockout, multi-factor enforcement), or to test credential stuffing against systems you own or have permission to test.

Risk & ethical notes (important for documentation)

- Both tools can be abused. Always explicitly document that tests were performed in an authorized lab environment or with written permission. Note potential impacts:
    - Hydra can lock accounts or trigger security controls on production services.
    - High-rate Hashcat use may consume GPU resources and energy; ensure lab safety and isolation.
- Include a short authorization statement in your report (who authorized, scope, targets, start/end times).

Example high-level commands (safe, non-actionable)

- Hashcat (conceptual):
    - `hashcat -m <mode> -a <attack> <hashfile> <wordlist or mask>`
    - Note: in documentation specify mode `1800` for sha512crypt (`$6$`) if relevant to your lab.
- Hydra (conceptual):
    - `hydra -L <userlist> -P <passlist> <target> <service>`

    - Note: in documentation emphasize throttling (`-t`), delays, and responsible usage to avoid lockouts.

(Do **not** include targeted credentials or commands against production IPs in public documentation.)

When to choose which — one-liners

- If you have hashes => **Hashcat** (offline, faster, GPU).
- If you are testing live authentication behavior or credential stuffing => **Hydra** (online, protocol-aware).
- If you want to evaluate both password strength and real-world attack surface => use them together in sequence in a controlled lab: extract hashes → attempt offline cracking with Hashcat → use known weak creds against live test services with Hydra (with explicit scope & rate limits).

Recommendations & Next Steps:

Enforce a strong password policy and deploy MFA immediately. Harden authentication endpoints by disabling password-based SSH access, enabling NLA for RDP, and implementing Fail2Ban with tuned thresholds for SSH/web logins. Replace any fast password hashes with Argon2id and centralize authentication logs to a SIEM for detection. Re-run the offensive tests after each change and record metrics (success rates and time-to-crack) to validate mitigation effectiveness. Finally, expand testing to include hybrid/hash-rule attacks and credential-stuffing simulations in a controlled environment to better emulate real adversaries